COGNITIONX EGYPT · 3 AUGUST 2026

AI-Native Development

From Writing Code to Owning the Full SDLC

> *A role-evolution talk, not a tools talk.*

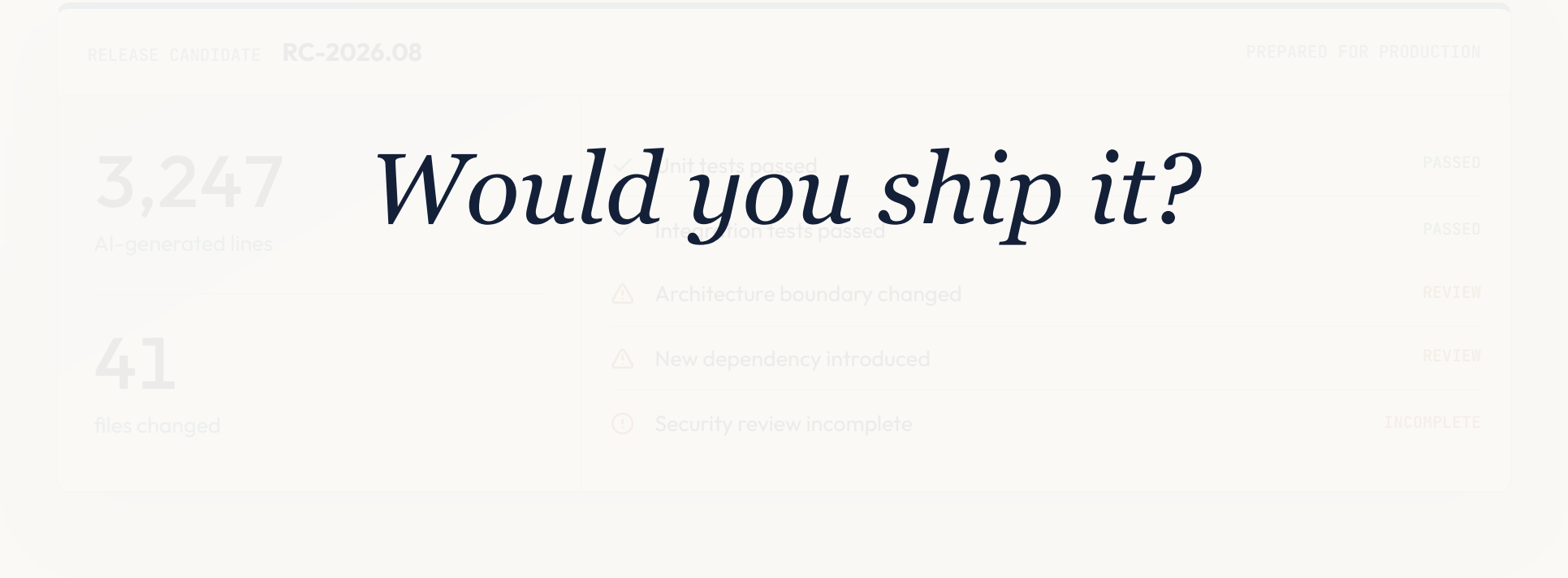
Direction · Constraints · Evidence · Outcomes

Hany Saad Senior Engineering Manager, ITWorx

Connect on LinkedIn

You didn't write this code.

But your name is on the release.



Would you ship it?

AI can generate the implementation.

The engineer still owns the outcome.

THE VERIFICATION SHIFT

The Bottleneck Moved

Generating code

is becoming easier.

→

Knowing whether it deserves to ship

is becoming harder.

The new bottleneck is **verification.**

Do not surrender control to AI.
Move your control to a higher level.

Code → Behavior → Architecture → Risk → Outcome

You are the orchestrator—not the typist.

HOW YOU BUILD CHANGES

AI in Development Is No Longer Optional

Use the right level of AI for the right task.

01 Vibe Coding

BEST FOR

Rapid prototypes, MVP experiments, demos, hackathons, idea validation, and disposable spikes.

> *Optimize for learning speed—not production confidence.*

EXAMPLES

Lovable | bolt.new | Google AI Studio

02 AI-Assisted Coding

BEST FOR

Daily coding, debugging, refactoring, tests, documentation, explanation, and unfamiliar code.

> *AI proposes; the engineer reads, integrates, and remains responsible.*

EXAMPLES

GitHub Copilot | Cursor | Equivalent IDE assistants

03 AI Coding Agents

BEST FOR

Well-specified issues, repository analysis, test repair, controlled modernization, and PR preparation.

> *Delegate the task—not the accountability.*

EXAMPLES

Claude Code | OpenAI Codex | Gemini CLI | Copilot coding agent

The question is no longer whether to use AI—but how to use it effectively, safely, and responsibly.

Tool examples illustrate categories and are not endorsements. Choose tools according to task, security, privacy, cost, and organizational policy.

SAME PROMISE. DIFFERENT CONDITIONS.

+55.8%

FASTER

Controlled, self-contained task

Peng et al. · Microsoft Research / arXiv · 2023

−19%

SLOWER

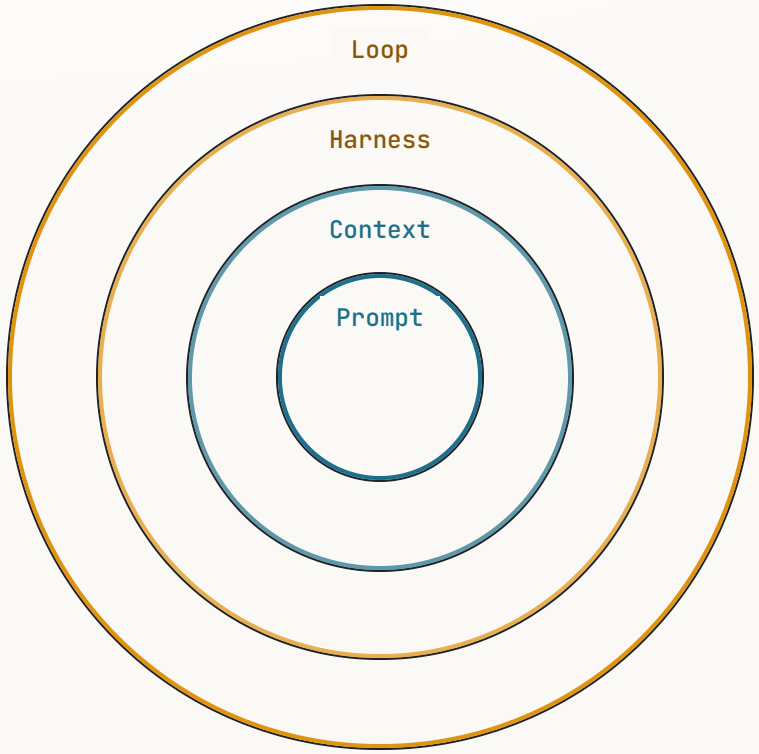
Experienced developers, mature repositories

METR · Mature repository study · 2025

Both results are real. What engineering conditions explain the difference?

A PRACTICAL WORKING MODEL

Prompt → Context → Harness → Loop

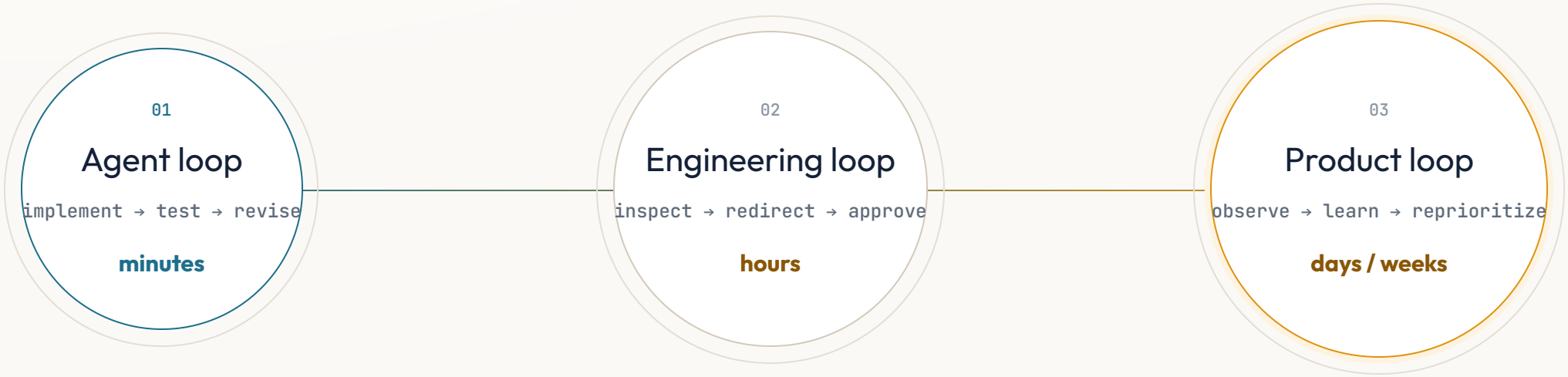


1	**Prompt**	What are we asking for?
2	**Context**	What does the agent need to know?
3	**Harness**	What can it access, execute, and change?
4	**Loop**	How does it verify, retry, stop, and escalate?

Each layer contains—and depends on—the one before it.

RESPONSIBILITY EXPANDS WITH LEVERAGE

Three Loops. Three Clocks.



The inner loop got fast. Your job moved outward.

THE DELIVERY SHIFT

From Sequential Delivery to a Continuous, Human-Guided Loop

The lifecycle remains. Execution accelerates. Human judgment moves to the control points.

TRADITIONAL DELIVERY

Sequential handoffs

- Humans create most artifacts manually
- Coding consumes much of the delivery cycle
- Testing and review follow implementation
- Documentation and operational readiness often lag
- Feedback arrives after release

AI-NATIVE DELIVERY

Continuous, human-guided loops

- Humans frame, specify, constrain, and approve
- AI accelerates bounded execution
- Tests and evidence are defined earlier
- Documentation and delivery artifacts evolve with the change
- Production feedback continuously updates the next cycle

The agent accelerates the work inside the lifecycle. The engineer still owns the lifecycle.

ACCOUNTABILITY HAS AN ADDRESS

What Owning the Full SDLC Actually Means

01	**Intent**	Problem, outcome, and the decision not to build
02	**Specification**	Acceptance criteria and edge cases
03	**Architecture**	Boundaries, contracts, and data flows
04	**Delegation boundary**	Scope, access, tools, and stopping conditions
05	**Evidence pack**	Tests, security, performance, and observability
06	**Release case**	Rollout, provenance, and rollback
07	**Production learning**	Telemetry, incidents, feedback, and improvement

Every artifact has an owner. AI is not that owner.

VALUE BEGINS BEFORE GENERATION

Start With Intent, Not Code

> *“Prevent session overbooking. If capacity is reached, offer a waitlist and notify organizers.”*
>
> Raw business request

- What counts as capacity?
- Who may override it?
- What personal data is exposed?
- What is the rollback behavior?
- Is check-in concurrent?
- How is the waitlist ordered?
- What happens when notification fails?

ACCEPTANCE CRITERIA Atomic capacity check · Authorized override · Stable waitlist order · Failure-tolerant notification · Auditable rollback

The engineer creates value before the first line is generated.

CHALLENGE THE PLAN BEFORE ACCEPTING THE WORK

Plan. Constrain. Delegate.

AGENT'S PROPOSED PLAN

01 Add waitlist storage

02 Update check-in endpoint

03 Add organizer notification

04 Update attendee UI

05 Add tests

HUMAN REVIEW

— Concurrency is missing

— Authorization boundary is unclear

— Notification failure must not reverse check-in

— No new dependency without approval

— Audit event required

Delegation contract `bounded tasks · affected interfaces · tests required · permissions allowed · stop / escalation conditions`

VERIFY THE WHOLE CHANGE

Verify the System, Not the Diff

Verify the behavior, boundaries, operations, and critical decisions—not only the generated code.

BUSINESS REQUIREMENT

Prevent session overbooking. If capacity is reached, offer a waitlist and notify organizers.

SOURCE OF TRUTH

CHANGE ASSURANCE	Evidence	What it proves
01 **Behavior**	Capacity + concurrency tests	→ **No overbooking under load**
02 **Boundaries**	Authorization + dependency / secret scans	→ **Access and dependencies stay constrained**
03 **Operations**	Migration / rollback + telemetry / alerts	→ **Deployments stay reversible and observable**
04 **Critical decisions**	Human review of critical logic	→ **Business rules remain intentional**

RELEASE DECISION

✓ ALL FOUR PROVEN

Release with controls

NAMED ENGINEER DECIDES

ANY EVIDENCE GAP

Hold or narrow scope

Green tests show execution. Change assurance shows the requirement survived.

CHANGE ASSURANCE STACK

Code Review Is Necessary. It Is No Longer Sufficient.

Do not stop reviewing code.

Stop pretending code review alone proves the system is safe.

The Engineer Moves Up the Stack

AI fluency builds on engineering maturity—not instead of it.

01	**Software Fundamentals**	Programming · OOP · data structures · APIs · databases · debugging · Git
02	**Engineering Practice**	Testing · code review · CI/CD · secure coding · observability
03	**System Design**	Architecture · boundaries · data contracts · reliability · trade-offs
04	**Product & Domain Judgment**	User needs · business rules · acceptance criteria · domain context
05	**AI Collaboration**	Task framing · context · constraints · tool use · critique
06	**End-to-End Ownership**	Verification · risk · orchestration · release decisions · outcomes

HOW CAPABILITY COMPOUNDS

01–02
Foundation
Build and ship sound software.

03–04
Maturity
Shape systems and problems.

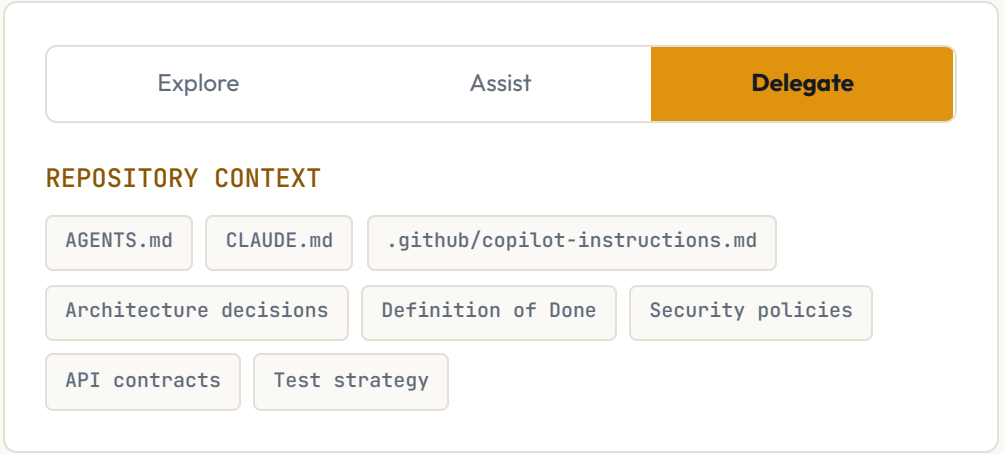
05–06
Leverage
Direct, verify, and own.

AI amplifies the stack.
It does not replace its foundation.

IMPROVE ONE WORKFLOW FIRST

Monday Morning: Five Moves

1. Choose one bounded, real task
2. Write acceptance criteria before invoking the agent
3. Make repository context explicit
4. Put automated evidence before human attention
5. Measure delivery—not generated lines



MEASURE DELIVERY cycle time · review effort · rework · escaped defects · change failure rate

OWN THE OUTER LOOPS

You are the orchestrator—not the typist.

The inner loop got fast. Own the outer loops.

AI-native development is not about producing more code.
It is about owning the system that turns intent into reliable outcomes.

Hany Saad
Senior Engineering Manager, ITWorx


LinkedIn


Presentation repo